



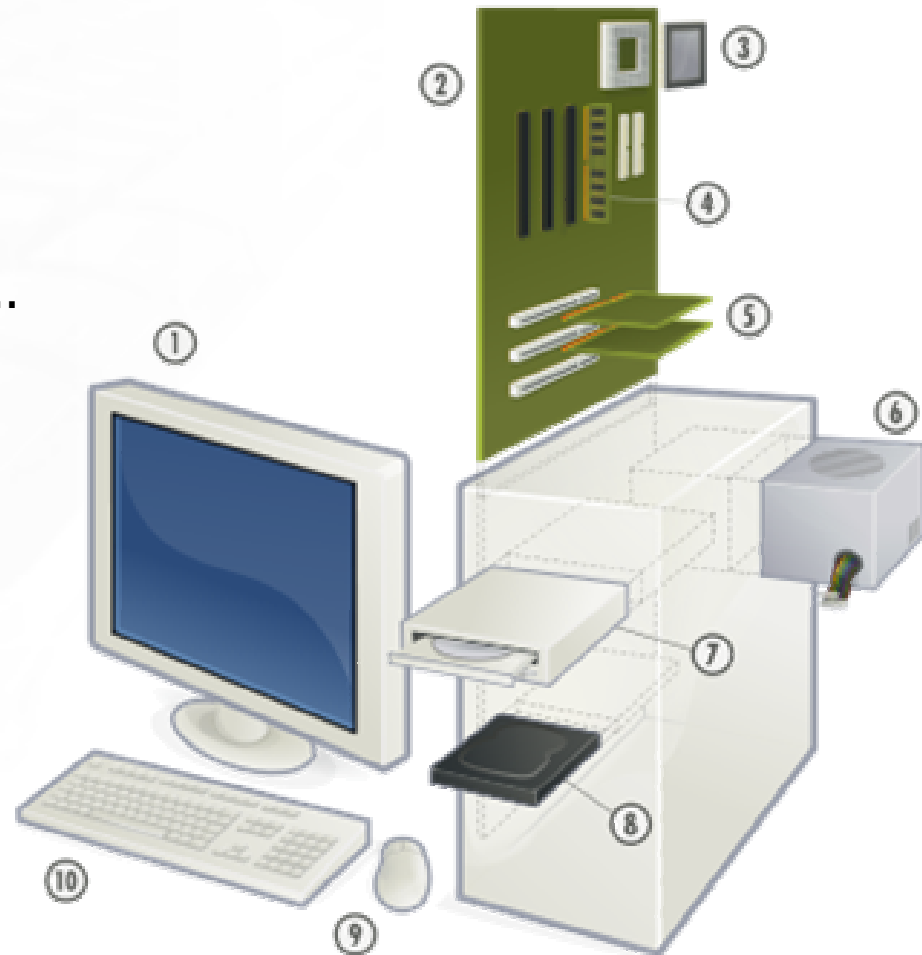
Introdução

(Aula 2)

Organização Estruturada de Computadores

Introdução – Arquitetura de Hardware

- 01- Monitor
- 02- Placa-Mãe
- 03- Processador
- 04- Memória RAM
- 05- Placas de Rede, Som, Vídeo, Fax...
- 06- Fonte de Energia
- 07- Leitor de CDs e/ou DVDs
- 08- Disco Rígido (HD)
- 09- Mouse
- 10- Teclado



Introdução – Conceitos (1)

■ Computador Digital

- É uma **máquina** que pode resolver problemas executando uma série de **instruções**.
- Máquina Programável.

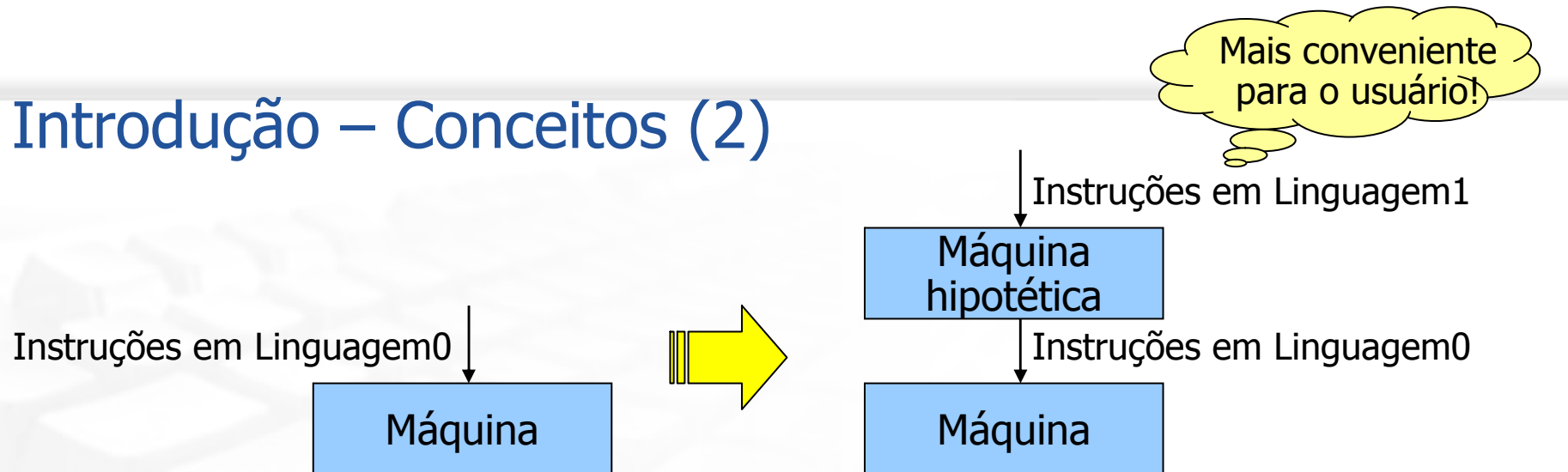
■ Programa

- Uma sequência de **instruções** que descrevem a maneira de se realizar uma determinada tarefa.

■ Linguagem de Máquina

- Conjunto de instruções básicas que os circuitos eletrônicos de um determinado computador pode reconhecer e executar diretamente
- Linguagem primitiva (binária), mas é a linguagem que a máquina realmente entende.
 - Somar 2 números
 - Comparar o valor de um número com zero.
 - Copiar um conjunto de dados de uma parte da memória para outra parte
- **Complicada para uso humano**

Introdução – Conceitos (2)



■ Organização Estruturada de Computadores

- Maneira estruturada de se organizar os computadores em diferentes **Níveis de Abstração**.
- Cada “abstração” é construída com base naquela que encontra-se imediatamente abaixo.
- Projeto de computadores de maneira sistemática e organizada.

Tradução x Interpretação ⁽¹⁾

■ Linguagens de alto nível

- C, C++, Delphi, Pascal, Fortran, Java,...
- Facilitam a comunicação dos seres humanos com a máquina, sendo linguagens mais próximas dos humanos.
- Devem ser convertidas em linguagem de máquina para serem executadas.
- Métodos de conversão: a **tradução** e a **interpretação**.

Tradução x Interpretação (2)

■ Tradução

- Programa de alto nível é primeiro **convertido** por completo para a linguagem de máquina para então ser executado.
- Em linguagens de alto nível, esta tradução é realizada pelo compilador (compilação ~ tradução)
- Ex: C, Pascal.

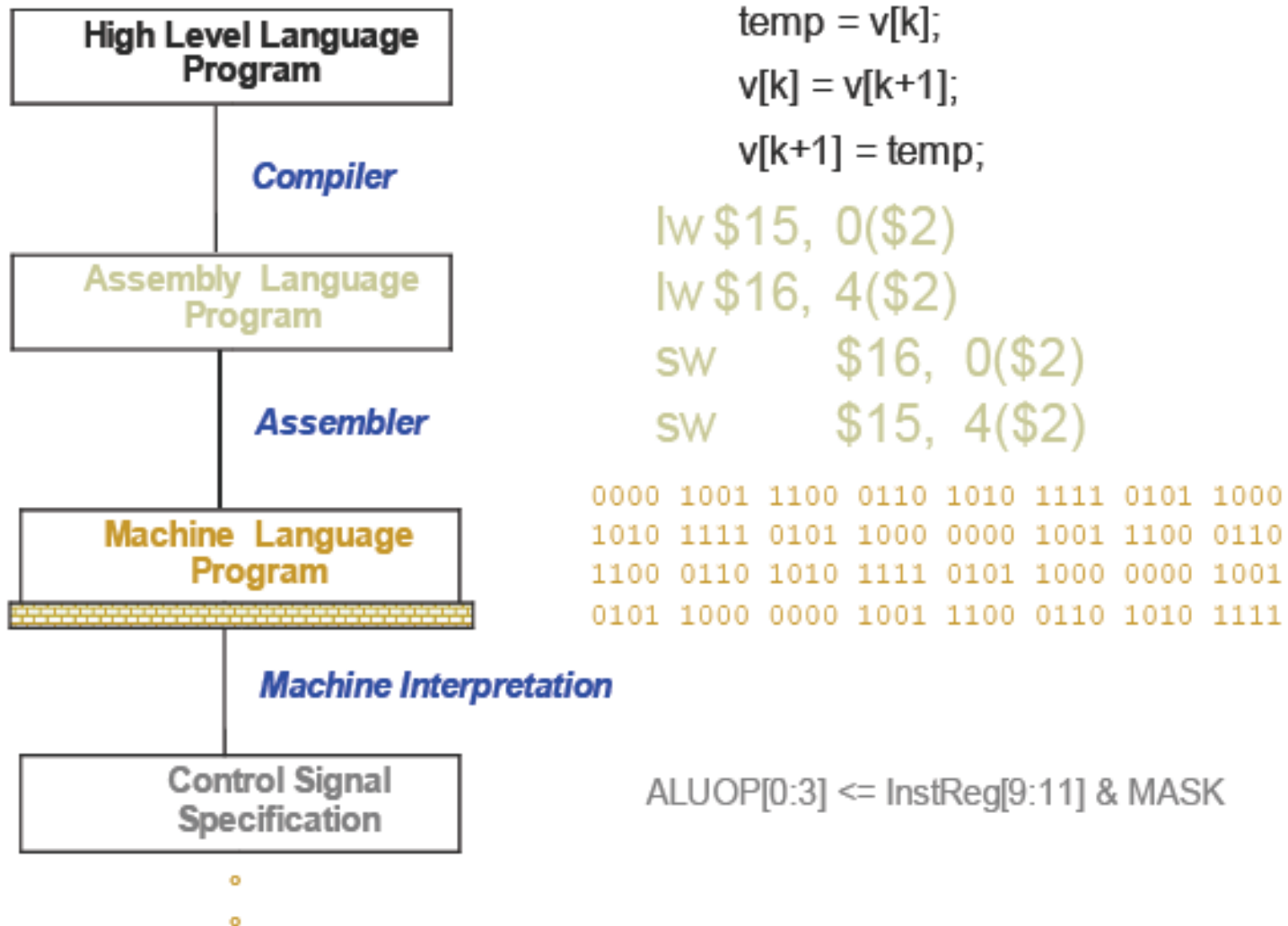
■ Interpretação

- Depois de cada instrução de alto nível ser examinada e decodificada, ela é executada imediatamente.
- Ex: Haskell.

■ Métodos Híbridos

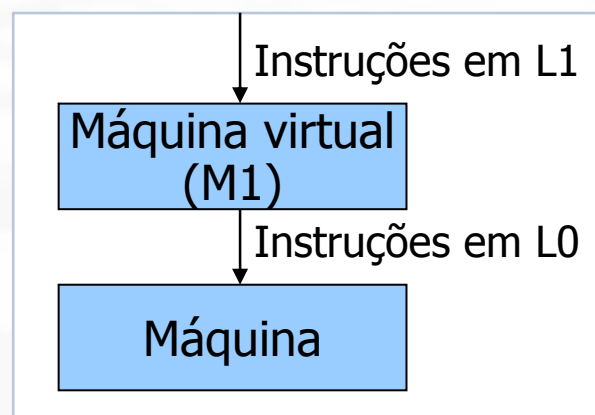
- Ex: Java

Tradução x Interpretação (3)



Máquina Virtual ⁽¹⁾

- Representa uma abstração capaz de reconhecer e executar diretamente as instruções de uma linguagem específica

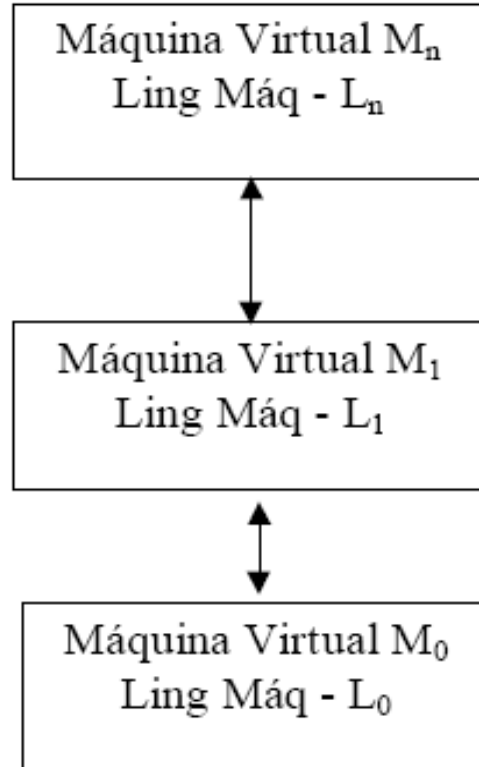


- Seja **L0** uma linguagem de máquina e **M0** a máquina capaz de executá-la
- Seja **L1** uma linguagem em um nível de abstração acima de L0.
M1 é portanto a máquina virtual associada a L1 (abstração)
 - Máquina (M1) hipotética, cuja linguagem de máquina é a linguagem L1.
 - Na prática, M1 representa um programa escrito em L0, sendo executado (interpretado) por M0
 - Os programas escritos em L1 são OU traduzidos OU interpretados por M1

Máquina Virtual

- Para que a tradução ou a interpretação sejam tarefas práticas, L0 e L1 não devem ser muito “diferentes”
- Máquinas Multi-Níveis
 - Diferentes camadas ou níveis de abstração.
 - Nível ou Camada: uma máquina (computador real ou virtual) e sua linguagem correspondente, sobre a qual uma nova camada pode ser acrescentada.
 - A linguagem ou o nível mais baixo é o mais simples, enquanto a linguagem ou o nível mais alto é o mais sofisticado.

Máquina de Vários Níveis (1)



Programas em L_n são interpretados (ou traduzidos) por um interpretador (ou tradutor) residente na máquina M_{n-1}

Programas em L_1 são interpretados (ou traduzidos) por um interpretador (ou tradutor) residente na máquina M_0

Programas em L_0 são interpretados e executados diretamente pelo hardware do computador (circuitos eletrônicos)

Máquina de Vários Níveis (2)

- Cada máquina virtual tem associada a si uma linguagem, composta de todas as instruções que essa máquina pode executar.

Uma máquina define uma linguagem.
Uma linguagem define uma máquina.

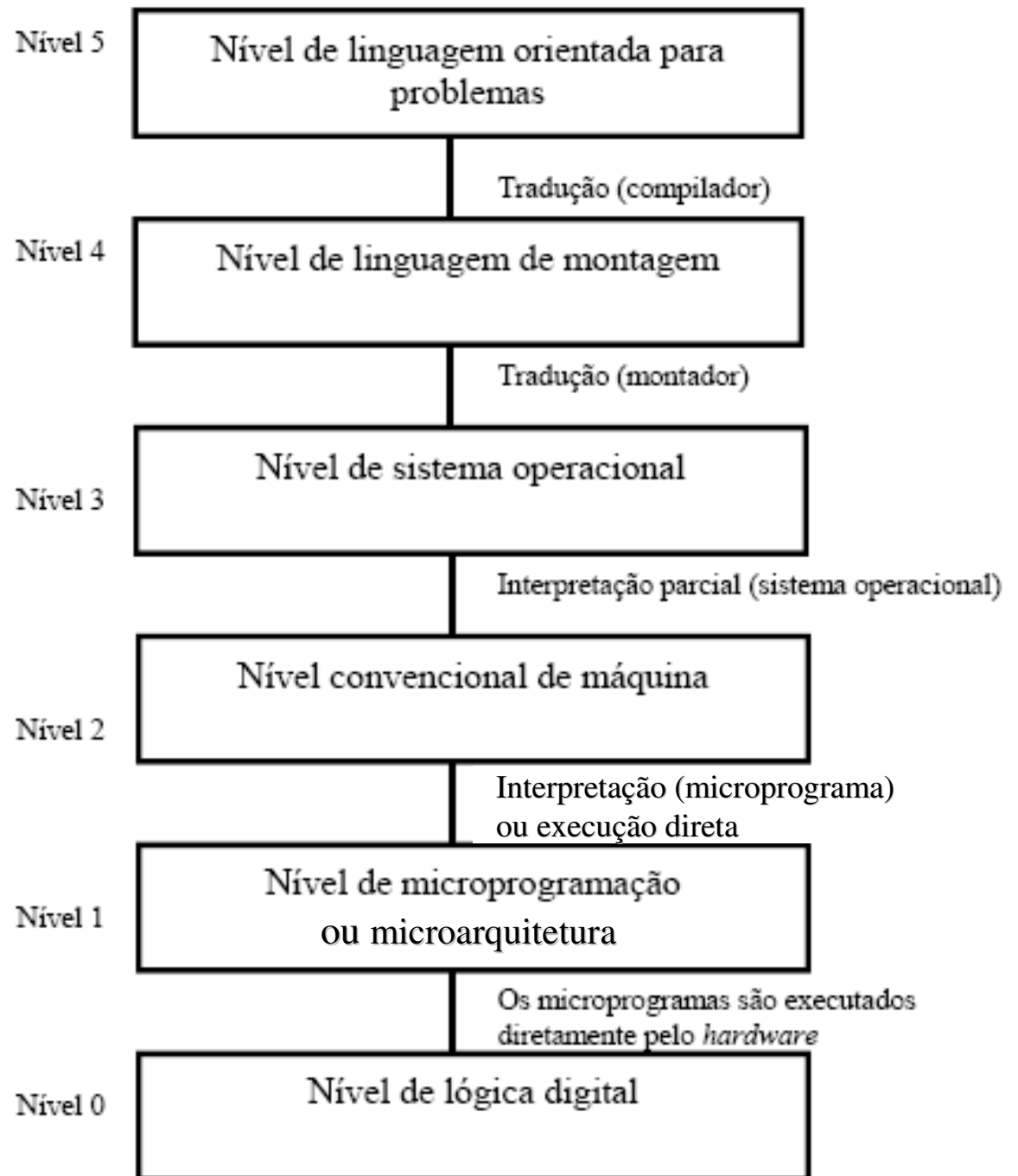
- Um computador com n níveis pode ser visto como n máquinas virtuais distintas.

Máquina de Vários Níveis ⁽³⁾

- Vantagens desta abordagem:
 - Um computador pode ser considerado como composto por "n" máquinas virtuais, cada uma delas com sua própria configuração e linguagem de máquina;
 - Uma pessoa, cujo trabalho seja gerar programas (aplicativos) para a máquina virtual de nível "k" ou outro qualquer, não precisa conhecer ou se preocupar com as particularidades dos níveis inferiores ao seu;
 - A estrutura de níveis permite ver o computador como um conjunto hierárquico de facilidades que possibilitam flexibilidade e independência ao usuário.

Máquina de Vários Níveis Modernas

“A maioria dos computadores modernos possui dois ou mais níveis, sendo que máquinas com seis ou mais níveis são cada vez mais comuns.”



Nível dos Dispositivos (-1???)

- Situado abaixo do nível 0
- Microeletrônica
- Características físicas
- Malha de transistores
- Tecnologias de fabricação de circuitos integrados

Nível 0 ou Nível da Lógica Digital

- É composto pelo hardware da máquina
- O nível 0 executa diretamente as instruções (ou microinstruções) submetidas pelo nível 1.
- **Portas Lógicas**
 - São os objetos de interesse dos projetistas de computadores nesse nível
 - Dispositivos digitais construídos a partir de componentes analógicos (Ex: transistores)
 - Ex: portas AND, OR e NOT
- Combinação de portas lógicas:
 - Funções aritméticas;
 - Memórias (registradores);
 - Processadores.

Nível 1 ou Nível da Microarquitetura ⁽¹⁾

- Nesse nível, inicia-se o conceito de programa como uma seqüência de instruções a serem executadas diretamente pelos circuitos eletrônicos.
- Enxerga-se:
 - Um conjunto de 8 a 32 registradores (memória local)
 - Um circuito chamado **ULA (Unidade Lógica e Aritmética)**
- Os registradores e a ULA são conectados para formar o **Caminho de Dados (*Data Path*)**, estrutura sobre a qual os dados fluem.
- A operação básica do caminho de dados consiste na seleção de um ou de dois registradores para que a ULA opere sobre eles

Nível 1 ou Nível da Microarquitetura (2)

- Em algumas arquiteturas, no nível 1 há um programa denominado **microprograma**
 - Função: interpretar as instruções de nível 2, executando-as em seguida
 - É o microprograma quem controla a operação do caminho de dados
- O controle do *data path* por software
 - Realizado geralmente em máquinas CISC (*Complex Instruction Set Computer*).
- Nas máquinas não microprogramadas, o caminho de dados é quase sempre controlado por hardware
 - Realizado geralmente em máquinas RISC (*Reduced Instruction Set Computer*).
- Poucas são as máquinas que têm mais de 20 instruções no nível da microarquitetura
 - A maior parte destas instruções envolve a movimentação de dados de uma parte da máquina para outra, ou alguns testes simples.

Nível 1 ou Nível da Microarquitetura (3)

■ Controle por software

- Microprograma = Interpretador (residente em memória ROM)
 - Busca, decodifica e executa as instruções, uma a uma, usando o caminho de dados para a realização de uma tarefa.
 - Exemplo: Execução de uma instrução de SOMA (ADD)
 - A instrução deve ser buscada na memória, seus operandos devem ser localizados e trazidos para os registradores, a soma deve ser calculada na ULA, e o resultado deve ser encaminhado para o lugar apropriado

■ Controle por hardware

- Os mesmos passos acima são executados, mas sem que haja um programa armazenado para controlar a interpretação das instruções

Nível 1 ou Nível da Microarquitetura ⁽⁴⁾

- Vantagens da microprogramação
 - Facilitar o projeto e a construção dos circuitos digitais
 - Implementação de parte da lógica digital dentro do firmware
 - Flexibilizar e possibilitar o desenvolvimento de instruções mais potentes a nível de máquina convencional.

Nível 2 ou Nível ISA

- **Nível ISA – *Instruction Set Architecture***
 - Ou Nível da Arquitetura do Conjunto de Instruções
- **Nível Convencional de Máquina**
- Define o conjunto das **Instruções** executáveis por uma máquina (processador)
 - Cada máquina ou processador tem sua linguagem própria de nível 2, chamada “**Linguagem de Máquina**” (documentada em manuais específicos de cada fabricante).
- Pode ser vista como a verdadeira interface entre o **software** e o **hardware**
- Discute-se:
 - Tipo de Dados
 - Modelos de Memória e de Endereçamento
 - Formato e Tipos de Instruções
 - ...

Nível 3 ou Nível do Sistema Operacional (SO) ⁽¹⁾

- Esse nível suporta um conjunto de novas instruções, uma organização diferente da memória, a capacidade de rodar dois ou mais programas de forma simultânea, e outros.
- Fornece serviços básicos para os níveis acima
 - Interface (gráfica ou linha de comando) com o usuário
 - Gerenciamento de memória
 - Escalonamento de processos
 - Acionamento de dispositivos de entrada e saída de dados, etc.
- Geralmente desenvolvido de forma **híbrida**
 - Existem instruções do nível 3 idênticas às de nível 2, que não são executadas (interpretadas) pelo sistema operacional

Nível 3 ou Nível do Sistema Operacional (SO) ⁽²⁾

- Níveis abaixo: dirigidos aos **programadores de sistema**
 - Níveis projetados para rodar interpretadores e tradutores (ex. compiladores)
 - Os programadores de sistema são especialistas em projetar e implementar novas máquinas virtuais/interpretadores/tradutores.
 - Predominância de Interpretação.
 - Linguagens freqüentemente numéricas, bom para as máquinas, mas ruim para as pessoas.
- Níveis acima: dirigidos aos **programadores de aplicação**
 - Programadores com problemas a serem solucionados.
 - Predominância de tradução (mas nem sempre).
 - Linguagens contendo palavras e abreviações (significativas para as pessoas).

Nível 4 ou Nível de Linguagem de Montagem

- Uma forma simbólica de representação das linguagens dos níveis mais baixos.
- Provê um método para as pessoas escreverem programas para os níveis 1, 2, e 3 de uma maneira não tão desconfortável.
 - Mnemônicos para as instruções de máquina.
- Os programas escritos em linguagem de montagem são primeiramente traduzidos para a linguagem dos níveis 1, 2 ou 3, e depois interpretados.
- **Montador:** programa que executa a tradução dos programas em linguagem de montagem para uma linguagem do nível 1, 2 ou 3.

Nível 5 ou Nível de Linguagens de Alto Nível

- Linguagens projetadas para serem utilizadas por programadores de aplicação com problemas a serem resolvidos.
- Ex. de linguagens de alto nível: C, C++, Basic, Java...
- Os programas escritos nessas linguagens são geralmente traduzidos para o nível 3 ou nível 4 por tradutores conhecidos como **compiladores**, embora às vezes sejam interpretados (como no caso de Java).

Mais Níveis ???

- Acima do nível 5 encontram-se coleções de programas projetados para criar máquinas especialmente adequadas para certas aplicações (ou domínios), contendo grandes quantidades de informação acerca da aplicação.
- Máquinas virtuais voltadas às aplicações
 - Administração, educação, projeto de computadores, realidade virtual, etc.
- Dependendo do projeto da arquitetura, os níveis podem variar

Arquitetura de Computadores

- Pontos fundamentais:
 - Computadores são projetados como uma série de níveis
 - Cada nível é construído em cima de seus precursores.
- Cada nível representa uma abstração distinta, com diferentes objetos e operações presentes
 - **Arquitetura do Nível:** conjunto de tipos de dados, instruções e características
- Abstrai-se o que é irrelevante, reduz a complexidade e foca no que interessa.

Arquitetura de Computadores:

é o estudo de como projetar as partes de um sistema de computador visíveis aos programadores.

Hardware, Software e Firmware ⁽¹⁾

■ **Hardware**

- É composto por objetos tangíveis (parte física) - circuitos integrados, placas de circuito impresso, cabos, fontes de alimentação, memórias, impressoras, etc.

■ **Software**

- É composto de instruções, algoritmos e por suas representações computacionais - os programas.

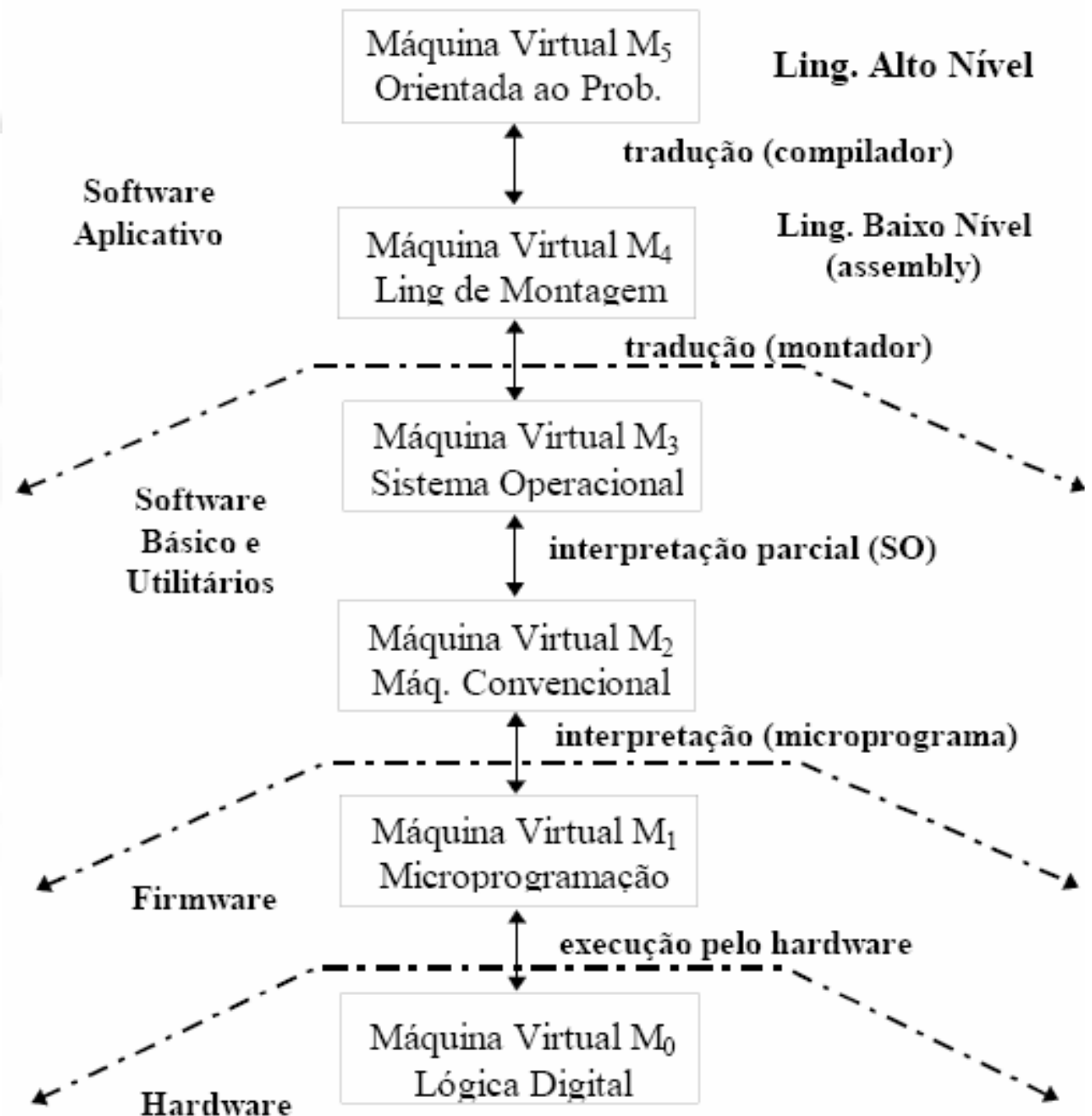
■ Hardware e Software são logicamente equivalentes.

- Qualquer operação efetuada pelo software pode também ser implementada pelo hardware... E qualquer instrução executada pelo hardware, pode também ser simulada pelo software

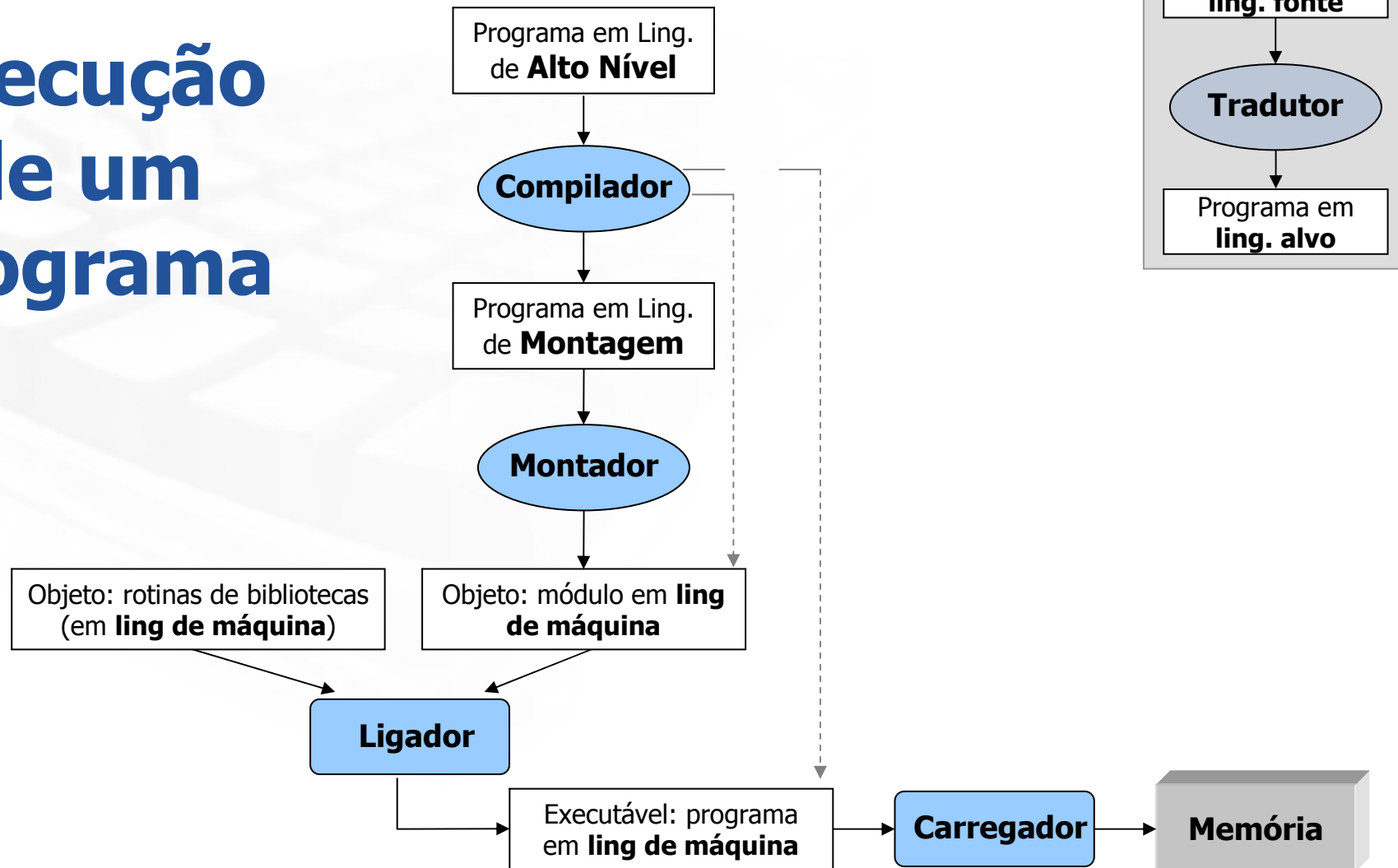
■ **Firmware**

- Software embarcado (embutido no dispositivo durante a fabricação)
- Controla o hardware diretamente. Ex.: BIOS (*Basic Input/Output System*).
- Em muitos computadores o microprograma está em firmware

Hardware, Software e Firmware (2)



Execução de um Programa



Compiladores, Montadores, Ligadores e Carregadores

(1)

■ **Compiladores**

- São programas que recebem como entrada arquivos texto contendo módulos escritos em linguagem de alto nível e geram como saída programa em linguagem de montagem (ou diretamente arquivos objeto) correspondentes a cada módulo.
- Se todas as bibliotecas ou módulos são apresentados como entrada, geram um programa executável diretamente.

■ **Montadores** (*Assemblers*)

- Montam um programa em linguagem de máquina a partir de sua versão em linguagem de montagem.
- Geram um **arquivo objeto**. Em geral, não pode ser executado diretamente pela máquina, por conter referências a sub-rotinas e dados especificados em outros arquivos (bibliotecas).

Ambos são TRADUTORES!

Compiladores, Montadores, Ligadores e Carregadores

(2)

■ **Ligadores (Linkers)**

- São programas especiais que recebem como entrada arquivos objetos e geram como saída o programa final em linguagem de máquina.
- Gera um programa executável a partir de um ou mais arquivos objeto.

■ **Carregadores (Loaders)**

- Para executar um programa, um loader deve ser utilizado.
- O carregador é, em geral, parte do sistema operacional.

Interpretadores

■ Interpretadores

- Recebem como entrada arquivos texto contendo programas em linguagem assembly, ou linguagem de alto nível, ou arquivos binários com instruções de máquina, e os executam diretamente.
- Interpretadores percorrem os programas, a partir de seu ponto de entrada, executando cada comando.
- Processadores são interpretadores implementados em hardware!

RISC x CISC

- CISC - *Complex Instruction Set Computer*
 - Arquitetura cujo processador é capaz de executar centenas de instruções complexas diferentes, sendo assim extremamente versátil.
 - Exemplos: 386, 486 da Intel.
 - Muitas das instruções guardadas no próprio processador.
- RISC - *Reduced Instruction Set Computer*
 - Uma linha de arquitetura de computadores que favorece um conjunto simples e pequeno de instruções.
 - Exemplos: SPARC, MIPS, PowerPC, DEC Alpha, etc.
 - Considerado mais eficiente e flexível que as CISC
 - As instruções tendem a ser executadas em poucos (ou mesmo um único) ciclos de relógio.
- Tamanho do código X Desempenho
 - Geralmente, o desempenho de um RISC é melhor do que de um CISC;
 - Código gerado por um RISC tende a ser mais longo e complexo.

Referências

- Andrew S. Tanenbaum, ***Organização Estruturada de Computadores***, Capítulo 1, 5ª edição, Prentice-Hall do Brasil, 2001.
- Lúcia Helena M. Pacheco, **Visão Geral de Organização Estruturada de Computadores e Linguagem de Montagem**. Universidade Federal de Santa Catarina. Centro Tecnológico, Departamento de Informática e de Estatística.

<http://www.inf.ufsc.br/~lucia/Arquivos-INE5607/Material%20anterior/OrganizacaoEstruturada170906.pdf>